

CAMPUSRESOLVE

**ONLINE CASE AND GRIEVANCE MANAGEMENT
SYSTEM**

A Community Engagement Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

By

Omkar Bagave

Seat No. of the Student

Under the esteemed guidance of

Name of the Internal Guide

Designation of Internal Guide

DEPARTMENT OF INFORMATION TECHNOLOGY

S.K. COLLEGE OF SCIENCE & COMMERCE

(Affiliated to University of Mumbai)

Plot No. 31, Sector 25, Seawoods, Navi Mumbai – 400706

Maharashtra

2026–27

Abstract

CampusResolve is a web-based Online Case and Grievance Management System developed to provide students with a clear and accountable way to report campus concerns, follow progress, communicate with responsible staff, and receive resolution updates. The platform replaces fragmented manual communication with a structured case workflow that records the reporter, subject, category, priority, status, department, assigned officer, evidence, comments, and resolution history.

The system uses a role-based model with four principal workspaces. Students create their own accounts and submit cases. Department teams review incoming work, assign cases, monitor service levels, and coordinate resolution. Case officers investigate assigned work and record progress or resolution remarks. Administrators manage users, roles, categories, departments, assignments, reports, audit trails, and system-wide configuration.

The project includes secure login, public student registration, password hashing, session-based access control, CSRF protection, upload validation, notifications, audit logging, escalation support, SLA tracking, and a MariaDB database. The application is built with PHP, MySQLi, MariaDB, JavaScript, Bootstrap, CSS, Docker, and shell-based deployment and testing tools. The interface has been designed with distinct layouts for personal tracking, department operations, officer investigation, and administrative control.

Keywords: Campus engagement; grievance management; role-based access; case tracking; PHP; MariaDB; audit trail.

ACKNOWLEDGEMENT

I take this opportunity to express my profound gratitude and indebtedness to my project guide for giving me the opportunity to accomplish this project. The guidance, feedback, and encouragement received during the planning and implementation of CampusResolve were valuable in shaping the final system.

I am thankful to the Principal and the faculty members of the Department of Information Technology, S.K. College of Science & Commerce, for their support and for providing an academic environment in which this project could be completed.

I also acknowledge the contribution of the teaching staff, laboratory assistants, classmates, and all individuals who helped identify practical campus grievance-management needs. Their observations influenced the role structure, case workflow, notification design, and emphasis on accountability.

Finally, I express my sincere thanks to my family for their continued support, motivation, and encouragement throughout the completion of this project.

DECLARATION

I, the undersigned, hereby declare that the project work titled “CampusResolve: Online Case and Grievance Management System” represents my own contribution carried out under the guidance of the internal project guide. The work has not been previously submitted to any other university or institution for the award of any degree or diploma.

Wherever reference has been made to the previous work of others, the sources have been acknowledged in the References section. I accept responsibility for the accuracy of the information presented in this report and for maintaining academic integrity in the development and documentation of the project.

I further declare that the system description, interface design, database design, implementation notes, and testing results in this report correspond to the project repository and its deployed application.

Certified by
(Ms./Mrs. Name of the Internal Guide)

Submitted by
(Ms./Mr. Omkar Bagave)

GUIDE INTERACTION DIARY FORM

No.	Date	Discussion / work completed	Guide initials
1	_____	Project problem, community context, and user roles identified.	_____
2	_____	Requirement analysis and case workflow reviewed.	_____
3	_____	Database schema and security controls discussed.	_____
4	_____	Role dashboards and user interface reviewed.	_____
5	_____	Testing, deployment, and final report reviewed.	_____

TABLE OF CONTENTS

1	Introduction	7
1.1	Purpose	7
1.2	Background Information	7
1.3	Scope of the Report	7
2	Literature Review	8
2.1	Existing Systems and Solutions	8
2.2	Role of Technology in Community Engagement	8
2.3	Review of Relevant Technologies	8
3	Methodology	9
3.1	Requirement Analysis	9
3.2	Data Collection Method	9
3.3	System Design	9
3.3.1	Use Case Design	9
3.3.2	Data Flow Design	9
3.3.3	Database and Entity Design	9
3.3.4	User Interface Design	10
3.4	System Testing	10
4	Observations and Analysis	11
5	Conclusion and Recommendations	12
5.1	Conclusion	12
5.2	Recommendations	12
6	References	13

1 Introduction

CampusResolve addresses a common institutional challenge: students may know that a concern should be reported, but they may not know where to submit it, which department is responsible, whether it has been assigned, or when a response can be expected. Staff members also need a consistent way to triage cases, assign ownership, record evidence, and demonstrate that actions were taken.

The system provides one authenticated web application for this workflow. A student submits a case through a guided form. The application stores the case in MariaDB, generates a reference number, and makes the case visible to the appropriate staff workspace. Staff users then process the case according to their role. Each important action can be recorded in the timeline and audit trail.

1.1 Purpose

The purpose of CampusResolve is to improve the accessibility, traceability, and responsiveness of campus grievance handling. The system aims to reduce dependence on informal messages, avoid lost requests, and give students meaningful visibility into the progress of their cases.

The project also demonstrates how a modest PHP and MariaDB application can provide structured role-based workflows without requiring a separate client application. Its deployment configuration supports local Docker use, a bundled database, persistent uploads, and deployment to a hosted environment.

1.2 Background Information

Traditional grievance handling often depends on paper forms, direct messages, or separate spreadsheets maintained by individual departments. These methods make it difficult to search previous cases, measure response time, confirm assignment, or provide a reliable history to the student. They also create privacy and accountability risks when case information is shared outside an access-controlled system.

CampusResolve treats a grievance as a case with a lifecycle. The lifecycle begins with registration and submission, continues through review, assignment, investigation, communication, and resolution, and ends with feedback and archival reporting. This model creates a shared understanding between the student and the institution.

1.3 Scope of the Report

This report describes the problem, project objectives, literature context, methodology, system design, implementation, testing, observations, conclusion, and recommendations. It focuses on the functioning web application and its SQL database. It does not claim that the current demonstration deployment replaces an institution's legal policies, official appeal process, or records-retention policy.

2 Literature Review

Digital grievance and case-management systems are commonly designed around three principles: a simple intake process, controlled ownership of work, and a permanent history of actions. A useful system must make reporting easy for the public while making processing disciplined for staff.

2.1 Existing Systems and Solutions

Existing institutional solutions range from email-based reporting to general-purpose service-desk platforms. Email is familiar, but it does not inherently provide structured fields, priority classification, assignment controls, or consistent reporting. General-purpose service-desk systems provide stronger workflow support, but they may be costly, difficult to customize, or too complex for a small educational institution.

CampusResolve takes a focused approach. It keeps the public submission experience simple while providing separate workspaces for students, department teams, officers, and administrators. The system stores structured data that can be filtered, audited, and used for reports.

2.2 Role of Technology in Community Engagement

Technology can support community engagement when it lowers the effort required to raise a concern and increases confidence that the concern has been received. A reference number, status history, notification, and visible ownership provide evidence that a submission is part of an institutional process rather than an isolated message.

The system must also respect privacy. Role-based access ensures that a student sees personal cases, an officer sees assigned cases, a department team sees its queue, and an administrator sees system-level information. This separation creates accountability without exposing unnecessary personal information.

2.3 Review of Relevant Technologies

PHP provides server-side request handling and session management. MariaDB stores normalized application data and supports relational constraints and indexes. JavaScript adds client-side validation, password visibility controls, responsive navigation, and asynchronous authentication. Bootstrap provides common responsive components, while the project's CSS layer supplies the CampusResolve visual system. Docker and Docker Compose simplify local setup by packaging the application, database initialization, and persistent storage. Shell scripts and GitHub Actions support repeatable checks. The application also uses password hashing, CSRF validation, upload MIME checks, and audit records as practical safeguards.

3 Methodology

The project followed an iterative software-development methodology. Each iteration moved from a concrete requirement to a database or interface change, followed by syntax checks, smoke tests, browser checks, and live verification.

3.1 Requirement Analysis

The requirements were derived from the needs of four user groups. Students require registration, secure login, case submission, evidence upload, status tracking, comments, notifications, and feedback. Department teams require queue visibility, filters, assignment, escalation awareness, and workload monitoring. Officers require an investigation workspace, status updates, remarks, and access to assigned evidence. Administrators require user and role management, categories, departments, reports, audit trails, and workflow configuration.

Non-functional requirements include maintainability, responsive layout, secure sessions, database persistence, clear error messages, and a deployable configuration. The interface must remain understandable for a first-time student while providing dense operational information to staff.

3.2 Data Collection Method

The project used a requirement-driven design method. The workflow was analyzed as a sequence of real institutional actions: a concern is raised, categorized, reviewed, assigned, investigated, updated, resolved, and evaluated. The supplied report reference also informed the structure of this documentation, while the supplied interface reference informed the login visual language.

The implementation was then inspected through repository tests, PHP syntax validation, browser automation, and live role checks. These checks confirmed that the application pages render, authentication redirects to the correct workspace, and the deployment serves the updated user interface.

3.3 System Design

3.3.1 Use Case Design

The principal actors are Student, Department Manager, Case Officer, and Administrator. A Student registers, logs in, creates a case, attaches evidence, comments, tracks status, and submits feedback. A Department Manager reviews the department queue, assigns an officer, changes priority, and monitors service levels. A Case Officer investigates assigned cases and records progress or resolution remarks. An Administrator manages institutional configuration and reviews audit and analytics data.

3.3.2 Data Flow Design

A case enters through the public authenticated student workflow. The server validates the session and request, applies CSRF checks, validates the submitted fields and upload type, writes the case to MariaDB, and produces a reference number. The department queue reads pending cases and the assignment workflow creates an ownership link. Updates produce timeline events and notifications. Reports read aggregated case and SLA data without changing the underlying history.

3.3.3 Database and Entity Design

The database is initialized from `database.sql` and is also available as the portable export `sql/campusresolve.sql`. The schema includes users, roles, departments, categories, complaints/cases,

assignments, attachments, comments, timelines, notifications, notification queue records, SLA policies, escalations, feedback, password-reset records, and audit logs. Foreign keys and indexes connect the records while preserving a clear case history.

3.3.4 User Interface Design

The four workspaces use a shared CampusResolve design system but different compositions. The student workspace emphasizes personal progress and calm status cards. The department workspace emphasizes queue scanning and operations. The officer workspace emphasizes focused investigation and action. The administrator workspace emphasizes system-wide statistics and governance.

The login page uses the supplied reference as its visual direction: a white rounded panel, compact navigation, a spacious form on the left, a teal curved visual area on the right, layered aqua waves, pill-shaped controls, and soft shadows. The same aqua language is used in the authenticated application without removing the role-specific visual distinctions.

3.4 System Testing

Testing included PHP syntax validation across the repository, static smoke checks, CSRF and upload guard checks, JavaScript syntax checks, browser authentication and theme checks, live staff-login checks, public registration checks, and live deployment checks. The final regression suite reported 50 PHP files without syntax errors and 21 successful browser assertions.

4 Observations and Analysis

The completed system demonstrates that a small role-based portal can provide a coherent case workflow when its database, access controls, and interface are designed together. The most important improvement is the relationship between a case and its history. A student can see the case reference and current status, while staff can see ownership, remarks, and audit context.

The four-workspace design also reduces cognitive overload. Students are not presented with administrative functions. Officers are not required to navigate system configuration. Department staff can concentrate on queue operations, while administrators receive a wider control view. This separation makes the interface easier to learn and reduces the likelihood of accidental changes.

The MariaDB database gives the project persistence beyond a static demonstration. Docker initialization and the SQL export make the deployment repeatable. The application can use its bundled database or connect to an externally managed database through environment variables. A persistent database volume remains necessary in production so that deployment replacement does not erase records.

The testing results indicate that the core workflow is operational, but a production institution would still need policy review, formal backup procedures, email-provider configuration, privacy review, and administrator training. Demonstration passwords must also be changed before real use.

5 Conclusion and Recommendations

5.1 Conclusion

CampusResolve provides a practical web-based foundation for managing student cases and grievances. It replaces informal reporting with a role-based workflow that supports registration, authentication, submission, routing, investigation, notifications, auditability, and resolution feedback. The application combines a MariaDB database with a responsive PHP interface and a deployment model that can be run locally or hosted.

The project also shows the value of designing for different responsibilities rather than presenting one generic dashboard to every user. A student needs reassurance and progress visibility. A department needs queue control. An officer needs an investigation workspace. An administrator needs oversight. The resulting system keeps a common visual language while giving each role an appropriate layout.

5.2 Recommendations

The next release should add institution-managed email delivery, because reliable email notifications will improve status communication when students are not actively logged in. The notification queue already provides a suitable foundation for this integration.

A production deployment should add scheduled database backups, tested restoration procedures, stronger secret management, HTTPS enforcement, rate limiting, and centralized monitoring. Seeded demonstration credentials should be rotated or disabled before public use.

The application could later include a mobile-friendly progressive web experience, richer analytics, multilingual labels, configurable appeal workflows, department-level service reports, and integrations with institutional identity providers. These additions should preserve the existing role boundaries and audit history.

6 References

1. PHP Documentation. **PHP Manual**. Available at: <https://www.php.net/docs.php>
2. MariaDB Documentation. **MariaDB Server Documentation**. Available at: <https://mariadb.com/kb/en/documentation/>
3. MDN Web Docs. **HTML, CSS, and JavaScript Documentation**. Available at: <https://developer.mozilla.org/>
4. Bootstrap Team. **Bootstrap Documentation**. Available at: <https://getbootstrap.com/docs/>
5. Docker Documentation. **Docker Compose Documentation**. Available at: <https://docs.docker.com/compose/>
6. OWASP Foundation. **OWASP Cheat Sheet Series**. Available at: <https://cheatsheetseries.owasp.org/>
7. GitHub. **GitHub Actions Documentation**. Available at: <https://docs.github.com/actions>
8. CampusResolve Project Repository. **Online Case and Grievance Management System**. Available at: <https://github.com/Shubh0o7/complaint-management-system>